

Introducción al desarrollo web en entorno servidor

Arquitecturas, servidores, protocolo HTTP y APIs: los cimientos antes de entrar en Java y Spring Boot.

Qué vamos a ver en esta unidad

01

Aplicaciones y arquitecturas web

Web vs escritorio, front-end / back-end, perfiles

02

Servidores web y de aplicaciones

Qué hace cada uno y con qué herramientas

03

Arquitecturas de software

Monolito, capas, microservicios, serverless...

04

El protocolo HTTP

Peticiones, respuestas, métodos, códigos y cabeceras

05

Servicios web y APIs

REST, GraphQL, WebSocket y un caso real

06

Este módulo en la práctica

De la teoría a Java y Spring Boot

01

BLOQUE 1

Aplicaciones y arquitecturas web



Aplicación de escritorio vs. aplicación web



Aplicación de escritorio

- **Instalación:** Se instala en cada equipo; posibles problemas de compatibilidad con el hardware o el sistema operativo.
- **Red:** Solo la necesita si la base de datos no es local.
- **Interfaz:** Depende del lenguaje y del sistema operativo en que se ha creado.



Aplicación web

- **Instalación:** Solo en el servidor; el cliente no instala nada.
- **Acceso:** Cualquier usuario con conexión y un navegador puede acceder.
- **Interfaz:** Siempre HTML, CSS y JavaScript (y frameworks derivados).

El desarrollo web actual

- **Campo en evolución constante** — abarca la creación y el mantenimiento de sitios y aplicaciones que operan a través de Internet.
- **No es solo funcionalidad** — también importa cómo se pone la aplicación a disposición de los usuarios: despliegue y paso a producción.
- **El objetivo es garantizar** — accesibilidad, estabilidad, escalabilidad y seguridad, con el menor tiempo posible hasta salir al mercado.



Time to market

Es el tiempo que transcurre entre decidir construir algo y ponerlo realmente en manos del usuario. Cuanto antes esté disponible una funcionalidad, antes empieza a aportar valor — por eso el despliegue es parte del desarrollo, no un paso aparte.

Front-end vs. back-end



Front-end (cliente)

- **Dónde se ejecuta:** en el navegador del usuario.
- **Con qué se construye:** HTML y CSS para estructura y estilo; JavaScript para la interactividad.
- **Qué permite:** animaciones, validaciones de formularios y actualizar contenido sin recargar la página (AJAX).



Back-end (servidor)

- **Dónde se ejecuta:** en el servidor web o de aplicaciones.
- **De qué se encarga:** la lógica de negocio, el acceso a bases de datos y la generación de contenido dinámico.
- **Cómo colaboran:** el servidor genera la respuesta y el cliente puede completarla de forma asíncrona.



Perfiles profesionales en el desarrollo web



Front-end developer

Diseño y maquetación con HTML, CSS y JavaScript (y sus frameworks). Cuida la presentación en cualquier dispositivo y el posicionamiento en buscadores.



Back-end developer

Lógica de negocio, acceso a datos y administración del servidor. Tecnologías: Java (Servlets/JSP, Spring), PHP, Python, ASP.NET, Ruby...



Full-stack developer

Conoce ambas partes sin ser necesariamente experto en una sola tecnología. Puede colaborar en cualquier capa del proyecto.

El back-end como servicio universal

- **La tendencia actual** — es que el back-end se vuelva universal: el servidor expone su funcionalidad a través de APIs que cualquier tipo de cliente puede consumir.
- **Un mismo back-end puede servir a** — aplicaciones web (SPA), apps móviles (iOS y Android), aplicaciones de escritorio y otros servicios o sistemas.
- **El resultado** — el back-end se desvincula de una interfaz concreta: ofrece datos en formatos estándar (JSON, XML) que cada cliente interpreta y representa a su manera.



Por qué importa

Este desacoplamiento es la base de la arquitectura de APIs REST que usaremos durante el módulo: un back-end en Spring Boot no sabe (ni le importa) si quien le pregunta es una web, un móvil o otro servicio.

Página web vs. aplicación web



Página web

- **Qué es:** un documento o conjunto de documentos mostrados en un navegador.
- **Estática:** su contenido no cambia salvo que alguien lo edite manualmente; puede verse incluso sin servidor.
- **Dinámica:** su contenido varía según interacciones, usuario o datos del servidor; ya necesita uno.



Aplicación web

- **Qué es:** una herramienta de software más compleja que combina páginas dinámicas (y estáticas) con tecnologías web.
- **Se parece a:** una aplicación de escritorio, pero se ejecuta en un navegador, lo que la independiza del sistema operativo cliente.
- **Siempre necesita:** un servidor web y, normalmente, otros componentes (base de datos, servidor de aplicaciones...).

Tecnologías más usadas en cliente y servidor

Perfil	Entorno	Tecnología	Usos comunes
Front-end / Cliente	Navegador web	HTML + CSS + JavaScript	Estructura, estilo e interactividad. Frameworks JS (React, Vue, Angular) para SPAs.
Back-end / Servidor	Servidor web + BBDD	Java (Servlets/JSP, Spring) · PHP · Python · .NET · Ruby	Lógica de negocio, acceso a datos, generación de páginas dinámicas y APIs.

Nuestro stack de back-end: Java + Spring

- **Lenguaje** — Java, con Spring y Spring Boot como framework principal.
- **Servidor de aplicaciones** — Tomcat, habitualmente embebido dentro de la propia aplicación Spring Boot.
- **Patrón de diseño** — MVC, tal y como veremos más adelante en esta misma unidad.
- **Formato de intercambio** — JSON, para construir APIs REST que consuman distintos tipos de cliente.



Todo lo que veas en este bloque —capas, MVC, APIs, HTTP— lo vas a tocar con código real en Java durante el curso.

02

BLOQUE 2

Servidores web y de aplicaciones

Servidor web vs. servidor de aplicaciones



Servidor web

- **Qué hace:** recibe peticiones HTTP/HTTPS de los clientes y devuelve recursos estáticos (HTML, CSS, imágenes, JavaScript).
- **Funciones:** gestiona las solicitudes y sirve contenido estático.



Servidor de aplicaciones

- **Qué hace:** proporciona un entorno de ejecución para aplicaciones web, gestionando la lógica de negocio y la comunicación con bases de datos.
- **Funciones:** procesa peticiones dinámicas (validar un login, generar un informe...) y ejecuta código del lado del servidor.

Ejemplos de servidores más habituales



Servidores web

Apache HTTP Server, Nginx y Microsoft IIS. Software libre y multiplataforma en el caso de Apache y Nginx; muy usados también como proxy inverso.



Servidores de aplicaciones Java

Apache Tomcat (contenedor de Servlets), WildFly / GlassFish (Java EE). Spring Boot incluye Tomcat embebido por defecto.



Otros entornos

Node.js, entorno de ejecución de JavaScript en el servidor. Útil para comparar cómo distintos lenguajes resuelven el mismo problema.

03

BLOQUE 3

Arquitecturas de software

Arquitecturas de software: visión general

- **Qué son** — modelos que describen cómo se relacionan y funcionan los distintos elementos que participan en el intercambio y procesamiento de información.
- **En el back-end** — existen varios estilos principales: monolítica, cliente-servidor, en capas, servicios web, microservicios, serverless, , SOA (Service-Oriented Architecture) y EDA (EDA (Event Driven Architecture)).
- **No hay una arquitectura "perfecta"** — la elección depende de los requisitos y objetivos de cada proyecto; las iremos viendo una a una.

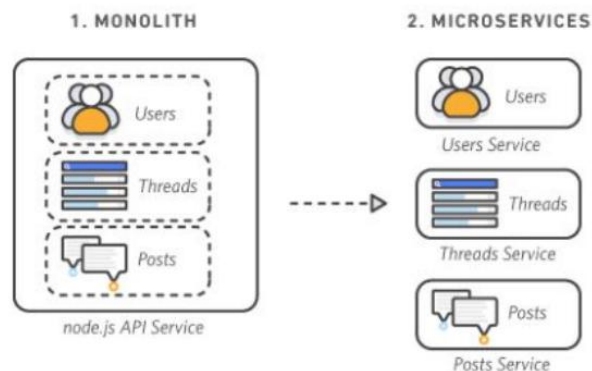


En esta unidad

Veremos 8 estilos arquitectónicos de forma introductoria. No hace falta dominarlos todos ahora: el objetivo es que reconozcas el vocabulario y entiendas qué problema resuelve cada uno.

Arquitectura monolítica

- **Qué es** — un enfoque tradicional en el que todos los componentes de la aplicación se agrupan en un solo bloque: interfaz, lógica de negocio y acceso a datos.
- **Ventaja principal** — es fácil de desarrollar y desplegar, sobre todo en proyectos pequeños.
- **Limitación principal** — se vuelve complejo y difícil de mantener a medida que la aplicación crece; escalar significa escalar todo el bloque.
- **Detalle técnico** — todos los componentes se ejecutan en el mismo proceso y comparten recursos; un cambio en una parte puede afectar a las demás.



Capa de acceso a datos

Interfaz de usuario, lógica de negocio y acceso a datos conviven dentro de la misma aplicación, que se conecta a una única base de datos.

Cliente-servidor: cómo se comunican



La interfaz vive en el cliente: muestra las opciones posibles y traduce las acciones del usuario en peticiones. **La funcionalidad vive en el servidor:** recibe la petición, la procesa y devuelve una respuesta.

Cliente-servidor: ventajas y desventajas

La funcionalidad reside en el servidor: recibe las peticiones del cliente y genera la respuesta. La interfaz está en el cliente, que muestra las opciones posibles y envía las acciones al servidor.



Ventajas

- Centralización del control: el servidor gestiona accesos, recursos e integridad de los datos.
- Escalabilidad: cliente y servidor pueden crecer por separado.
- Portabilidad: el navegador independiza la app del sistema operativo cliente.
- Fácil mantenimiento: se puede actualizar el servidor sin apenas afectar a los clientes.



Desventajas

- Congestión del tráfico si muchos clientes envían peticiones simultáneas.
- Fallo del servidor: si cae, ninguna petición se satisface.
- Puede requerir software y hardware específico, aumentando el coste.
- Se mitiga con balanceo de carga, caché, réplicas y clústeres de alta disponibilidad.

Arquitectura de capas y modelo de 3 niveles



Arquitectura de capas

- **Idea:** divide la aplicación en capas físicas o lógicas, cada una con una responsabilidad específica.
- **Se comunican:** cada capa habla solo con la capa adyacente, a través de interfaces bien definidas.
- **Ventaja:** mejora la modularidad: se puede cambiar una capa sin afectar a las demás.



Modelo de 3 niveles

- **Presentación (cliente):** lo que el usuario ve: HTML, CSS y JavaScript en el navegador.
- **Aplicación (servidor):** las reglas y procesos de negocio; ejemplo: validar stock, calcular descuentos.
- **Datos (base de datos):** donde se guarda la información: usuarios, productos, ventas...

El modelo de 3 niveles, de un vistazo



Patrón Modelo-Vista-Controlador (MVC)

- **Importante** — MVC no es una arquitectura, es un patrón de diseño para organizar el código dentro de la capa de aplicación.
- **Modelo** — representa los datos y la lógica que los maneja. Ejemplo: una clase Usuario con sus atributos y métodos.
- **Vista** — lo que se muestra al usuario. Ejemplo: una plantilla que muestra el perfil del usuario.
- **Controlador** — hace de intermediario entre la Vista y el Modelo: recibe la petición, pide los datos al Modelo y se los pasa a la Vista.



Spot the spoiler

Spring Web MVC —el módulo que usaremos para construir controladores en Java— toma su nombre precisamente de este patrón. Lo verás aplicado con anotaciones (`@Controller`, `@Service`, `@Repository`) más adelante en el curso.

Arquitecturas orientadas a servicios



Servicios web

Componentes independientes que se comunican mediante protocolos estándar (HTTP). Cada servicio se desarrolla, despliega y escala por separado.



Microservicios

Evolución de los servicios web: los componentes se dividen en piezas aún más pequeñas y autónomas, cada una centrada en una tarea concreta.



Serverless

El proveedor cloud gestiona toda la infraestructura; el equipo solo escribe código. Escalado automático y pago por uso, ideal para funciones cortas.

SOA y arquitectura orientada a eventos (EDA)

- **SOA — Service-Oriented Architecture** — los componentes se organizan como servicios independientes y reutilizables que se comunican a través de interfaces bien definidas. Promueve interoperabilidad y flexibilidad.
- **EDA — Event Driven Architecture** — los componentes se comunican mediante eventos: uno los emite cuando ocurre una acción significativa y otros se suscriben para reaccionar.
- **En común** — ambas buscan desacoplar componentes. EDA es especialmente útil en sistemas distribuidos donde la comunicación asíncrona mejora escalabilidad y resiliencia.

Arquitecturas de un vistazo

Arquitectura	Qué resuelve	Cuándo elegirla
Monolítica	Simplicidad de desarrollo y despliegue.	Proyectos pequeños o equipos reducidos.
Cliente-servidor	Separar interfaz y lógica de negocio.	Prácticamente cualquier aplicación web.
Capas / 3 niveles	Organizar el código por responsabilidad.	Aplicaciones de tamaño medio-grande.
Microservicios	Escalar y desplegar partes de forma independiente.	Sistemas grandes con equipos separados.
Serverless	Eliminar la gestión de infraestructura.	Funciones puntuales y picos de demanda variables.
SOA / EDA	Reutilización de servicios y desacoplamiento por eventos.	Integración entre muchos sistemas distintos.

Buenas prácticas: principios SOLID

- **S — Responsabilidad única** — una clase debe tener una, y solo una, razón para cambiar.
- **O — Abierto/cerrado** — las entidades deben estar abiertas a extensión pero cerradas a modificación.
- **L — Sustitución de Liskov** — un objeto de una subclase debe poder sustituir a uno de la superclase sin romper el programa.
- **I — Segregación de interfaces** — ningún cliente debe depender de métodos que no usa.
- **D — Inversión de dependencias** — los módulos de alto nivel no deben depender de los de bajo nivel; ambos dependen de abstracciones.

S

Lo retomaremos

Son principios de diseño orientado a objetos, no exclusivos del desarrollo web. Los volveremos a ver con ejemplos en Java cuando trabajemos con Spring, donde la inyección de dependencias (la "D" de SOLID) es central.

Patrones de diseño

- **Qué son** — soluciones probadas y reutilizables para problemas recurrentes en el diseño de software.
- **De creación** — Singleton, Factory Method, Abstract Factory, Builder — cómo crear objetos de forma flexible.
- **Estructurales** — Adapter, Decorator, Composite, Proxy — cómo componer clases y objetos.
- **De comportamiento** — Observer, Strategy, Template Method, Command — cómo interactúan y se comunican los objetos.
- **Arquitectónicos** — MVC, arquitectura en capas, microservicios — estructura y organización a gran escala.

04

BLOQUE 4

El protocolo HTTP



Qué es HTTP, y HTTP vs. HTTPS



¿Qué es HTTP?

- **Definición:** protocolo de transferencia de hipertexto que permite la comunicación entre un cliente (navegador) y un servidor web.
- **Sin estado:** cada petición es independiente; la sesión se debe crear en la propia aplicación web (cookies, tokens...).
- **Se usa siempre:** desde cargar una página hasta una petición asíncrona que no cambia de pantalla.



HTTPS: la versión segura

- **Qué añade:** cifrado mediante SSL/TLS sobre la misma comunicación HTTP.
- **Por qué importa:** sin cifrar, los datos viajan en texto plano y son vulnerables a ataques "man-in-the-middle".
- **Qué garantiza:** confidencialidad, integridad y autenticación de los datos transmitidos.

Petición y respuesta HTTP

Request line

1

Método + ruta del recurso + versión del protocolo.

```
POST /?id=1 HTTP/1.1
```

Cabeceras (headers)

2

Metadatos sobre la petición o la respuesta.

```
Host: miweb.com  
Content-Type: application/json
```

Cuerpo (body)

3

Datos enviados (POST/PUT) o el recurso devuelto por el servidor.

```
{ "usuario": "pepe" }
```

```
POST /?id=1 HTTP/1.1
```

Request line

```
Host: www.swingvy.com  
Content-Type: application/json; charset=utf-8  
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.12; rv:53.0)  
Gecko/20100101 Firefox/53.0  
Connection: close  
Content-Length: 136
```

Header

```
{  
  "status": "ok",  
  "extended": true,  
  "results": [  
    {"value": 0, "type": "int64"},  
    {"value": 1.0e+3, "type": "decimal"}  
  ]  
}
```

Body message

Métodos (verbos) HTTP

Método	Para qué se usa
GET	Obtener o recuperar un recurso. No suele llevar cuerpo; los parámetros van en la URL (query string).
POST	Crear un nuevo recurso o enviar datos. Los datos van en el cuerpo, no son visibles en la URL.
PUT	Actualizar o reemplazar por completo un recurso existente.
PATCH	Modificar parcialmente un recurso existente.
DELETE	Borrar un recurso o entidad específica del servidor.
HEAD	Igual que GET pero sin el cuerpo de la respuesta; útil para comprobar si un recurso existe.

Códigos de estado HTTP

- **1XX · Informativas** — la petición ha sido recibida y el proceso continúa.
- **2XX · Éxito** — la acción se recibió, entendió y aceptó. Ejemplo: 200 OK.
- **3XX · Redirección** — el cliente necesita una acción adicional; el recurso se ha movido.
- **4XX · Error del cliente** — la petición contiene un error. Ejemplos: 403 Forbidden, 404 Not Found.
- **5XX · Error del servidor** — el servidor falló al completar una petición aparentemente válida.

#

Un número, tres dígitos

El código de estado va siempre en la primera línea de la respuesta, junto a la versión del protocolo y un texto descriptivo: por ejemplo, HTTP/1.1 200 OK.

Cabeceras HTTP más habituales



En la petición

- **Accept:** formato en el que el cliente quiere los datos (ej. application/json).
- **Host:** dominio al que se dirige la petición.
- **Content-Type:** formato de los datos enviados en el cuerpo.
- **User-Agent:** información sobre el navegador del cliente.



En la respuesta

- **Content-Type:** formato de los datos devueltos; crucial para que el navegador los interprete bien.
- **Content-Length:** tamaño en bytes de los datos devueltos.
- **Cache-Control:** durante cuánto tiempo se pueden cachear los datos.
- **Server:** información del software del servidor.

De una URL a un archivo en el servidor

Un servidor web escucha por defecto en el puerto 80 (443 para HTTPS). Una URL se descompone en protocolo, dirección, puerto y recurso:

`http://www.miweb.com:80/dir1/a.html`

protocolo	dirección	puerto	recurso
http	www.miweb.com	80	dir1/a.html
http://www.miweb.com:80/dir1/a.html			

Si no se indica un archivo, **el servidor sirve index.html** del directorio correspondiente (por defecto, /var/www/html en un servidor Linux con Apache).



Enlaces absolutos

```
<a href="http://miweb.com/a.html">
```

Incluyen la URL completa. Independientemente de dónde esté la página, siempre se accede a esa web concreta.



Enlaces relativos

```
<a href="dir2/a.html">
```

Indican solo la ruta. El navegador la busca en el mismo servidor y directorio donde está la página actual.

05

BLOQUE 5

Servicios web y APIs



Qué es un servicio web (API)



Servicio web / API

- **Qué es:** un conjunto de reglas y protocolos que permite a otra aplicación comunicarse remotamente para acceder a un servicio.
- **Cómo se identifica:** cada funcionalidad expuesta suele tener un endpoint (una URL) en el servidor.
- **Qué aporta:** integración entre sistemas, escalabilidad y reutilización: un mismo servicio puede tener varios clientes.



Página dinámica vs. servicio web

- **Página web dinámica:** genera HTML pensado para que lo vea una persona en el navegador.
- **Servicio web (API):** expone datos y funcionalidades para que los consuman otras aplicaciones, normalmente en JSON.
- **En arquitecturas modernas:** una SPA puede consumir varios servicios web para construir su interfaz.

Tipos de API más usados hoy



REST

Estilo arquitectónico que usa los métodos HTTP para acceder a recursos identificados por URLs. Intercambia datos normalmente en JSON. Es el que usaremos en el módulo.



GraphQL

Lenguaje de consulta para APIs: el cliente pide exactamente los datos que necesita, ni más ni menos, reduciendo el tráfico.



WebSocket

Conexión persistente y bidireccional entre cliente y servidor, ideal para comunicación en tiempo real (chats, notificaciones).

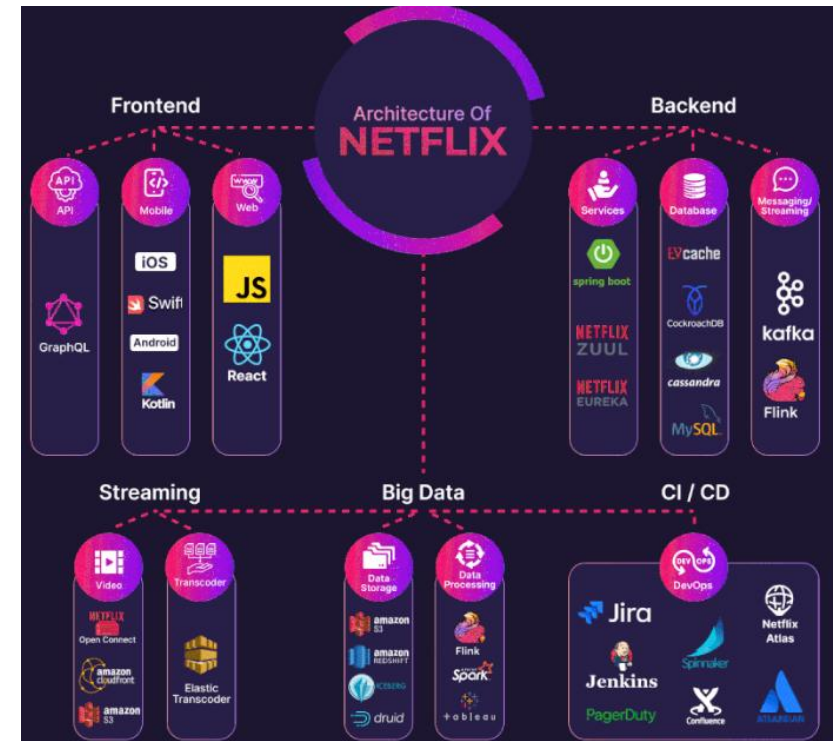
Ejemplo de arquitectura: Netflix

- **Escalable y resiliente** — diseñada para manejar grandes volúmenes de tráfico garantizando disponibilidad y rendimiento.
- **Basada en microservicios** — las distintas funcionalidades se dividen en servicios independientes que se comunican entre sí.
- **Cada microservicio se centra en una tarea** — lo que da más flexibilidad, escalabilidad y facilidad de mantenimiento a nivel individual.
- **Back-end en Java** — buena parte de los servicios de Netflix están contruidos sobre Spring Boot, el mismo framework que usaremos en este módulo.



Dato curioso

Netflix es uno de los ejemplos más citados en el sector de arquitectura de microservicios funcionando a gran escala en producción.



Lo esencial de esta unidad

- Una aplicación web separa front-end (cliente) y back-end (servidor), comunicados por HTTP.
- Existen muchos estilos de arquitectura de software; la elección depende del proyecto, no hay una única "correcta".
- MVC es un patrón de diseño para organizar el código, no una arquitectura en sí misma.
- HTTP define métodos (GET, POST, PUT, DELETE...), códigos de estado y cabeceras para estructurar peticiones y respuestas.
- Un servicio web (API) expone datos y funcionalidades a otras aplicaciones, no páginas para personas.



A PARTIR DE AQUÍ

Este módulo, en la práctica

Todo lo visto hoy —arquitecturas, MVC, HTTP y APIs— lo aplicaremos con Java como lenguaje y Spring Boot como framework:

1

Tomcat embebido

cada aplicación Spring Boot arranca con su propio servidor de aplicaciones, sin instalación aparte.

2

Arquitectura en capas + MVC

controladores, servicios y repositorios organizan el código igual que hemos visto hoy.

3

APIs REST

los controladores expondrán endpoints que responden en JSON a peticiones GET, POST, PUT y DELETE.

4

Buenas prácticas

SOLID y patrones de diseño aparecerán de forma natural al escribir servicios en Spring.